

Platform Engineering Technical Interview

Level: JL4 – Engineer

Sheet: JL4-2026-06-30-134819

Candidate name: Hristiyan Panoski

Interviewer: _____

Instructions:

- **Section A** (10 questions, 1 mark each): circle the letter of the correct answer.
- **Section B Part 1:** answer **both** questions in the space provided.
- **Section B Part 2:** answer **all four** questions in the space provided.

Section A – Multiple Choice

10 questions, 1 mark each. Circle the letter of the correct answer.

1. Looking at the Azure DevOps board, a new engineer sees her Tasks nested under larger items and wonders how her small pieces of work fit into the wider plan. In that hierarchy, your tasks connect upward to...
 - A. Only your laptop
 - B. Nothing
 - C. The payroll
 - D. A User Story / Feature they help deliver
2. On a tricky bug, a junior engineer is paired with a senior who shares her screen and talks through every decision out loud. Reflecting afterwards on why the team encourages this, pair-programming with a senior engineer mainly helps you...
 - A. Avoid work
 - B. Skip testing
 - C. Learn faster and improve code quality
 - D. Hide mistakes
3. Setting up branch policies, a lead configures the repository so that Pull Requests (PRs) can only complete with a rebase and fast-forward merge, rejecting the default merge-commit option. A new engineer, used to merge commits elsewhere, asks why the team is so particular about how branches land on main. Why does the team require a rebase & fast-forward merge?
 - A. It avoids running CI
 - B. It deletes the work item link
 - C. It allows skipping approvals
 - D. It keeps a linear, traceable history
4. An engineer pulls the next Task off the board and finds the requirement is vague - it is not obvious what 'done' would even look like, and the acceptance criteria are thin. Tempted to just start coding something plausible, they pause to consider the most professional move. You pick up a Task whose requirement is unclear. Best action?
 - A. Close it as done
 - B. Wait for the next status meeting
 - C. Guess and build something
 - D. Clarify it against Definition of Ready before starting

5. A new engineer joins the Platform Engineering team at a chip foundry and reads the coding standards on day one, where every reviewer keeps stressing 'clean, testable code.' During onboarding, their mentor asks them to pick the option that captures that team value. Which best describes the clean, testable code the team expects? (1 mark)
- A. Defers all tests until release
 - B. Mixes I/O and business logic for speed
 - C. Small, single-responsibility units with logic separated from side effects
 - D. Relies on global mutable state
6. A team refactoring a wafer-data parser adopts the Functional Core / Imperative Shell (FC/IS) pattern, reorganizing the module into two distinct layers. A junior engineer wiring up the new structure wants to be sure exactly what belongs in the inner layer before moving code around. In the FC/IS pattern, the 'core' should be... (1 mark)
- A. The user interface
 - B. The CI pipeline
 - C. Where file and network I/O happen
 - D. Pure functions with no side effects
7. While tracing how their new rule compiler reads a file, an engineer keeps seeing a first stage labelled the lexer, or tokenizer, that runs before anything else. Curious about what that stage actually does, she checks her understanding of the basics. A lexer/tokenizer... (1 mark)
- A. Executes the program
 - B. Draws the AST
 - C. Deploys code
 - D. Breaks input text into tokens
8. A senior walks a new hire through the rule compiler and shows two tree structures built from the same input: a parse tree, also called a concrete syntax tree, produced directly by the parser, and a separate Abstract Syntax Tree (AST) consumed by later stages. The new hire wonders why the tool bothers keeping two different trees. How does an AST differ from a parse (concrete syntax) tree? (1 mark)
- A. They are identical
 - B. The AST stores characters
 - C. The AST omits incidental syntax (punctuation) and keeps meaning
 - D. The AST has more nodes always
9. Reviewing a layout, a checker flags an enclosure rule on a structure where two stacked layers do not relate the way the deck demands. The layout engineer, comfortable with spacing and width rules, has rarely dealt with this third family of constraints that ties one layer to another. Her mentor asks her to state precisely what such a rule demands. An enclosure rule requires... (1 mark)

- A. One layer to surround another by at least a minimum amount (e.g., metal over contact)
 - B. Shapes to be deleted
 - C. Two shapes to overlap exactly
 - D. Layers to be merged
10. A reviewer points an engineer at pytest's parametrize feature after seeing a cluster of very similar tests, and the engineer, new to it, asks what parametrize is actually meant for. pytest's parametrize is used to... (1 mark)
- A. Mock the database
 - B. Run the same test over multiple inputs
 - C. Generate coverage
 - D. Deploy to many environments

Section B – Written Questions

Support each answer with a worked example or codebase reference – unsupported answers earn half marks at most.

Part 1 – Answer BOTH questions.

1. During a new-node kickoff, a customer engineer repeatedly hears the foundry team refer (8 marks)
to the technology's 'ground rules' as the reference point for the whole design effort.
Curious what specifically sits behind that label, he asks a process contact to define the
term. What are ground rules?
2. An engineer at the foundry has just pushed what they believe is a finished Task, but (8 marks)
the Continuous Integration (CI) run comes back red with two failing tests. A reviewer
is available right now and it is tempting to hand it over and sort the failures out in
parallel. Before doing anything rash, the engineer thinks through the proper sequence.
You finished coding a Task but tests are failing in CI. Walk through what you do before
requesting review.

Part 2 – Answer ALL four questions.

1. Reviewing a data structure whose fields are all public and modified from everywhere, (6 marks)
lead suggests the author lean on encapsulation instead. The author, fairly junior, wants to explain the concept and its payoff before reworking the type. What is encapsulation and why is it useful?

Encapsulation uses private fields(data), that are isolated from the rest of the code. Changes to data are made using functions/methods defined inside the class. The functions can include validation logic, or restrictions to data handling. This prevents outside code modifying object values, instead it offers a safe, controlled way to handle data. Encapsulation separates the internal implementation of the logic from the external user interface. Additionally the methods in encapsulation can give a safe way of working with data, and blocking unwanted or wrong type data.

https://github.com/PanoEth/Interview_GF/tree/main/part2/q1

2. An engineer picks up a ticket for a Bug where a malformed config silently produced (8 marks)
wrong output, and their lead is keen that this failure never recurs unnoticed. Before diving in, they discuss how tests should drive the whole fix. You're fixing a Bug. How do you use tests in the process?

3. Halfway through what looked like a small Task, an engineer keeps discovering ext (10 marks)
cases and dependencies that were never in the original ticket, and the work is quietly ballooning. He feels the temptation to just push on and absorb it all silently. What do you do when a task's scope grows mid-flight?

4. While extending the rule grammar, an engineer finds that the same expression is in (10 marks)
terpreted differently by two versions of the tool, and the team suspects the grammar itself is at fault rather than the code. A senior asks him to explain what it means for a grammar to be ambiguous and why that is something to avoid. What makes a grammar ambiguous, and why is that a problem?

Grammar becomes ambiguous when the same input can be analysed in multiple ways, leading to different outcome/results from exactly the same input in the two versions of the tool.

Different tools leading to different results outcomes is a problem since it adds:

Uncertainty - since the same input would lead to different answer, the code becomes uncertain since the final user wouldn't know which answer to trust (no "correct" answer).

Unpredictability - since the outcome depends on which version of tool is chosen, predicting the outcome is harder.

Difficulty in debugging - if there are wrong answers, its harder to debug since you can't tell if problem comes from the code logic or how the data is analysed.

https://github.com/PanoEth/Interview_GF/tree/main/part2/q4